Detecting Spam Messages with Natural Language Processing

Preprocessing, text encoding and supervised classifiers evaluated from small to large message collections

Project report | DLBAIPNLP01 - Project: NLP | Task 2: Text Classification for Spam Detection

Yousef Mohamed Mahmoud Mahmoud Alashmawy
9 October 2026

Code repository: https://github.com/yousefalashmawi/spam-detection-nlp

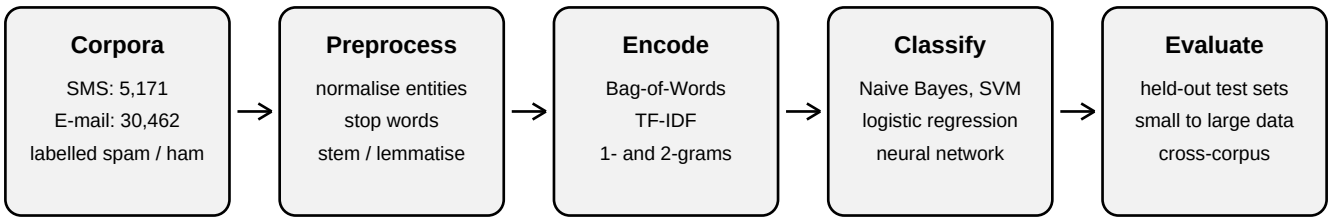
Abstract

Spam messages are unsolicited bulk messages that waste attention and can carry phishing links or malware. This project develops a natural language processing (NLP) system that classifies text messages and e-mails as spam or legitimate (ham). Two public corpora were used: the SMS Spam Collection with 5,171 unique messages as the small scenario and Enron-Spam with 30,462 unique e-mails as the large scenario. Messages were normalised, stop words were removed, words were stemmed or lemmatised, and the text was encoded as Bag-of-Words or TF-IDF n-gram vectors. Naive Bayes, a linear support vector machine (SVM), logistic regression and a small feed-forward neural network were trained and evaluated on held-out test data, with learning curves from 100 training messages up to the full corpora and tests across the two message types.

Replacing numbers, phone numbers, URLs and currency symbols with placeholder tokens was the most useful preprocessing step for SMS, raising F1 by 2.8 points for the linear models, while preprocessing hardly changed e-mail results. Linear models matched or beat the neural network at a fraction of its training time. Models trained on one message type failed on the other, so the final system routes each message to a channel-specific model. On untouched test data the SMS model reached 98.94% accuracy and flagged 2 of 904 legitimate messages; the e-mail model reached 99.20% accuracy with an F1 score of 0.992.

Graphical abstract

Fig. 1 Graphical abstract



Result: channel-specific models. SMS accuracy 98.94%, e-mail accuracy 99.20%; legitimate messages flagged: 0.22% (SMS), 1.04% (e-mail).

Source: Own representation of the system developed in this project. Figures in the bottom line are measured on the held-out test sets (see Section 4).

1. Introduction

Text classification assigns a document to one of a set of predefined categories; spam detection is its classic binary case, in which every incoming message is labelled as spam or ham (Jurafsky & Martin, 2013). A filter is only useful if it removes most spam while almost never hiding a legitimate message, because a lost personal or business message costs the user far more than one unwanted advertisement. This project builds such a filter and examines each step of the NLP pipeline in different scenarios. It addresses four questions:

Q1 Which preprocessing steps (normalisation, stop-word removal, stemming, lemmatising) and which text encoding (Bag-of-Words or TF-IDF) improve classification?
Q2 How does performance develop from a small to a large training set?
Q3 Do probabilistic, discriminative and neural classifiers differ in accuracy and cost?
Q4 Does a model trained on one kind of message generalise to another kind?

1.1 Related work

Naive Bayes classifiers have long been a standard approach to spam filtering. Metsis et al. (2006) compared several Naive Bayes variants on e-mail and, for that purpose, assembled the Enron-Spam corpus, which combines legitimate mail from Enron employees with collected spam. For mobile messages, Almeida et al. (2011) published the SMS Spam Collection and reported results of established classifiers as baselines, noting that short, abbreviation-rich SMS texts are harder to filter than e-mails. More recent work uses deep learning: Elakkiya et al. (2021) combine textual features with an attention mechanism to detect spam in social media, and Huan et al. (2022) join convolutional and bidirectional long short-term memory layers so that a text classifier captures both local n-gram patterns and longer-range word order. Such models are more expressive but need more data and computation. This project therefore compares classical linear models on sparse n-gram features with a neural network on the same features, and asks whether the extra cost is justified for this task.

1.2 Data collection

The task suggests the UCI Spambase data set. Spambase, however, contains only 57 pre-computed numeric features such as word and character frequencies for 4,601 e-mails, not the e-mail text (Hopkins et al., 1999). Stop-word removal, stemming and text encoding, which the task asks to evaluate, cannot be applied to it. Two public corpora with raw text and labels were therefore used (Tab. 1). The SMS Spam Collection (Almeida & Hidalgo, 2011) provides the small scenario. Enron-Spam (Metsis et al., 2006), in the CSV version prepared by Wiechmann (n.d.) with subject and body per e-mail, provides the large scenario. Exact duplicate texts were removed before any split; otherwise the same message could appear in both training and test data and inflate the scores.

Tab. 1 Data sets after de-duplication

Corpus	Type	Raw	Unique	Spam share	Median length	Train / test
SMS Spam Collection	SMS	5,574	5,171	12.6%	12 words	4,136 / 1,035
Enron-Spam	E-mail	33,665	30,462	47.8%	154 words	24,369 / 6,093

Source: Own representation of the loaded data; data from Almeida and Hidalgo (2011) and Metsis et al. (2006) via Wiechmann (n.d.). Raw counts exclude e-mails without any text.

The two corpora differ strongly. SMS spam is a minority class (about one message in eight) and SMS texts are short and informal, whereas Enron-Spam is almost balanced and its e-mails are more than ten times longer. This difference is used deliberately in Q4.

2. Method

2.1 Preprocessing

Five cumulative preprocessing levels were implemented (Tab. 2) so that the effect of each step can be measured separately. Stop words come from the NLTK English list (Bird et al., 2009). Stemming uses the Porter algorithm, which strips suffixes by rules (Porter, 1980); lemmatising maps words to dictionary base forms with WordNet (Miller, 1995), first as nouns and then as verbs. The placeholder tokens of the clean level are not stemmed.

Tab. 2 Preprocessing levels

Level	Operation	Example output for a spam SMS
raw	Lower-casing only (baseline)	free entry in 2 a wkly comp to win fa cup final tkts 21st may ...
clean	URLs, e-mail addresses, phone numbers, currency symbols and numbers replaced by tokens; punctuation removed	free entry in tok_num a wkly comp to win fa cup final tkts st may ...
stopwords	clean + NLTK English stop words removed	free entry tok_num wkly comp win fa cup final tkts st may ...
stem	stopwords + Porter stemming	free entri tok_num wkli comp win fa cup final tkt st may ...
lemma	stopwords + WordNet lemmatising	free entry tok_num wkly comp win fa cup final tkts st may ...

Source: Own representation of the implemented preprocessing (src/preprocess.py); example message from the SMS Spam Collection (Almeida & Hidalgo, 2011).

2.2 Encoding and classifiers

Preprocessed text was converted into sparse vectors of unigrams and bigrams that occur in at least two training messages, limited to 50,000 features. Two weightings were compared: raw counts (Bag-of-Words) and TF-IDF with sublinear term frequency, which down-weights terms that occur in many documents (Manning et al., 2008). The encoders were fitted on training data only. Four supervised classifiers from scikit-learn (Pedregosa et al., 2011) were trained on these vectors (Tab. 3).

Tab. 3 Classifier settings

Classifier	Type	Settings
Multinomial Naive Bayes	Probabilistic, generative	Additive smoothing alpha = 0.1
Linear SVM	Discriminative, maximum margin	C = 1, balanced class weights
Logistic regression	Discriminative, probabilistic	C = 10, liblinear solver, balanced class weights
MLP (neural network)	Feed-forward, deep learning baseline	One hidden layer of 64 ReLU units, Adam, early stopping, at most 50 epochs, 20,000 input features

Source: Own representation of the implementation (src/models.py), based on Pedregosa et al. (2011).

2.3 Evaluation protocol

Each corpus was split once into a stratified 80% training and 20% test part (random seed 42). The test parts were never used for training or for choosing a configuration. Spam is the positive class. Reported metrics are accuracy, precision (share of flagged messages that are spam), recall (share of spam that is caught), F1 (harmonic mean of precision and recall), ROC-AUC and the false-positive rate, the share of legitimate messages wrongly flagged. Accuracy alone is misleading on the imbalanced SMS data: a filter that lets every message through is already about 87% accurate.

Four experiments answer Q1 to Q4. **E1** trains every combination of preprocessing level, encoding and classical classifier on each training set. **E2** trains on stratified random subsets from 100 messages to the full training set, five different subsets per size, and always tests on the same full test set. **E3** compares all four classifiers with 5-fold cross-validation on the training data and on the test set. **E4** trains the best

classifier on SMS only, e-mail only and both combined and tests it on both test sets. Finally, for each channel the preprocessing level and classifier were selected by cross-validated F1 on the training data, refitted and evaluated once on the test set. The experiments ran in Python 3.13 with scikit-learn 1.9 and NLTK 3.10 on a Linux machine with two CPU cores; the executed notebook in the appendix shows all code and outputs.

3. Results

3.1 Effect of preprocessing and encoding (E1)

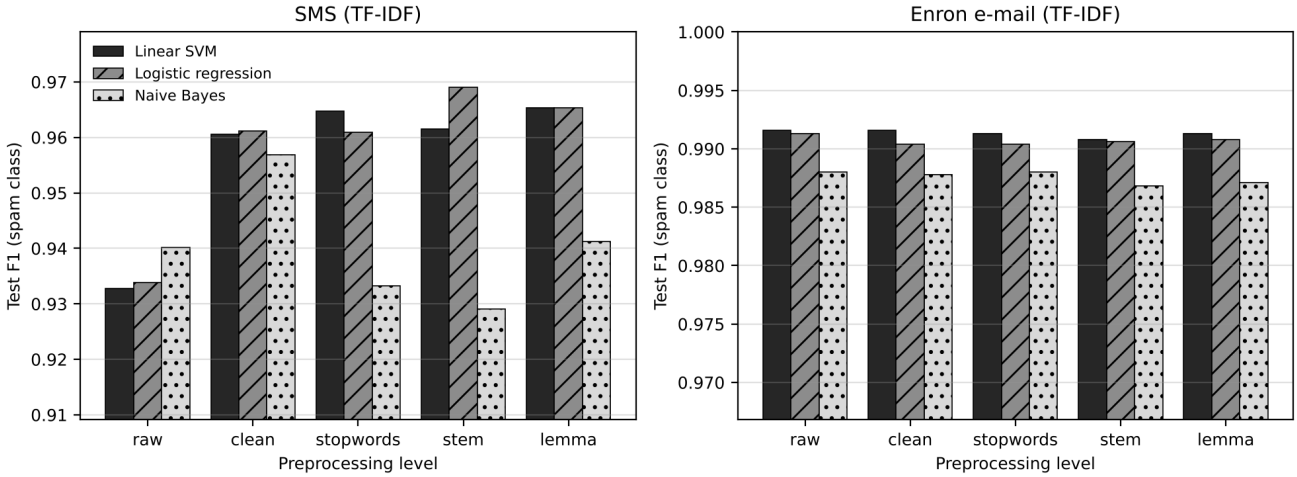
Tab. 4 Test F1 by preprocessing level (TF-IDF encoding)

Corpus	Level	Naive Bayes	Linear SVM	Logistic regression
SMS	raw	0.9402	0.9328	0.9339
SMS	clean	0.9569	0.9606	0.9612
SMS	stopwords	0.9333	0.9647	0.9609
SMS	stem	0.9291	0.9615	0.9690
SMS	lemma	0.9412	0.9653	0.9653
E-mail	raw	0.9880	0.9916	0.9913
E-mail	clean	0.9878	0.9916	0.9904
E-mail	stopwords	0.9880	0.9913	0.9904
E-mail	stem	0.9868	0.9908	0.9906
E-mail	lemma	0.9871	0.9913	0.9908

Source: Own representation of measured experiment outputs (results/e1_preprocessing.csv). Bag-of-Words results are in the same file.

For SMS, normalisation was the decisive step: replacing numbers, phone numbers, currency symbols and URLs by tokens raised F1 from 0.9328 to 0.9606 for the SVM and by 2.8 points on average for the two linear models (Tab. 4, Fig. 2). SMS spam typically asks the reader to call a premium number, text a short code or claim a cash prize; before normalisation, every number is a different rare word, afterwards the shared pattern becomes one strong feature. Stop-word removal, stemming and lemmatising changed the linear models by at most about one point, in both directions. They harmed Naive Bayes: removing stop words lowered its F1 by 2.4 points. In very short texts, function words such as 'you', 'your' or 'to' carry part of the persuasive style of spam, and Naive Bayes, which weighs every word independently, loses more from removing them. On e-mail, all levels lie within 0.1 points of each other; long texts contain enough evidence for the classifier to ignore noise. TF-IDF outperformed raw counts on average (SMS 0.952 vs. 0.950; e-mail 0.990 vs. 0.983).

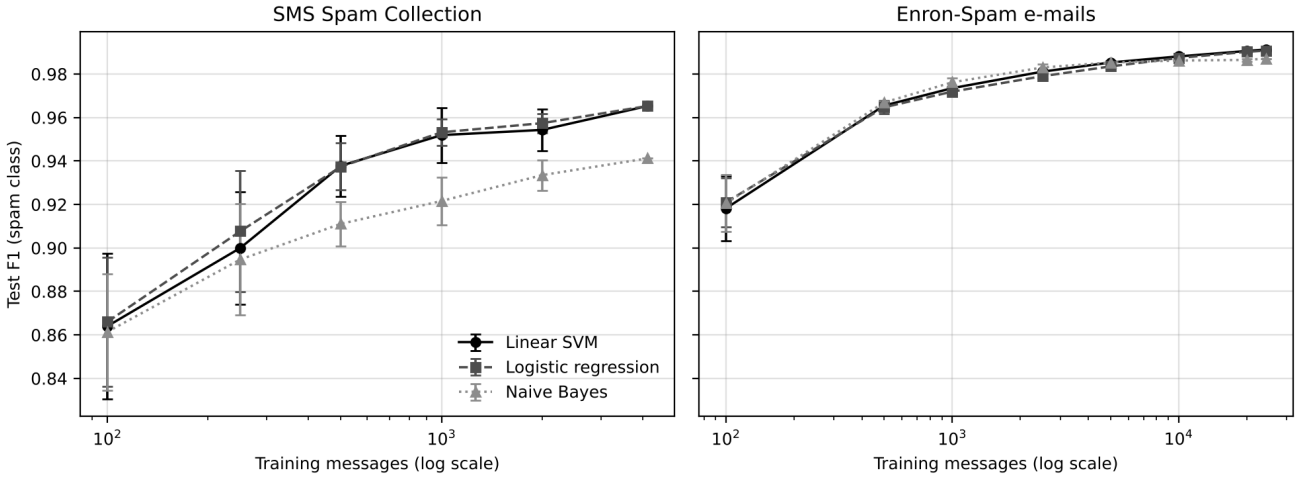
Fig. 2 Test F1 by preprocessing level



Source: Own representation of measured experiment outputs. Note the different vertical scales.

3.2 From small to large training sets (E2)

Fig. 3 Learning curves



Source: Own representation of measured experiment outputs (results/e2_learning_curve.csv). Points are means over five random training subsets, bars one standard deviation; the full training set is a single run.

Performance rose steeply up to about 1,000 training messages and then levelled off (Fig. 3). With 100 messages the SVM reached an F1 of 0.864 on SMS and 0.918 on e-mail; with the full training sets (4,136 and 24,369 messages) it reached 0.965 and 0.991. The spread between random subsets shrank from about 3 F1 points at 100 SMS messages to about 1 point at 2,000, and on e-mail to 0.2 points from 500 messages onwards, so small training sets give not only lower but also less reliable results. The false-positive rate on e-mail fell from 7.8% to 1.1%. Naive Bayes was competitive with little data but improved least as data grew, which matches the usual behaviour of a generative model with strong independence assumptions compared with discriminative models. The SMS curves are still rising at the full corpus size, so more labelled SMS data would probably help; the e-mail curves have almost converged.

3.3 Comparison of classifiers (E3)

Tab. 5 Classifier comparison (lemmatised text, TF-IDF)

Corpus	Classifier	CV F1 (mean ± sd)	Accuracy	Precision	Recall	F1	AUC	Train time
SMS	Naive Bayes	0.940 ± 0.018	0.9855	0.9677	0.9160	0.9412	0.9937	0.2 s
SMS	Linear SVM	0.945 ± 0.017	0.9913	0.9766	0.9542	0.9653	0.9978	0.2 s
SMS	Logistic regression	0.946 ± 0.017	0.9913	0.9766	0.9542	0.9653	0.9975	0.1 s
SMS	MLP	0.922 ± 0.025	0.9874	0.9917	0.9084	0.9482	0.9944	4.6 s
E-mail	Naive Bayes	0.987 ± 0.001	0.9877	0.9870	0.9873	0.9871	0.9989	16.0 s
E-mail	Linear SVM	0.990 ± 0.001	0.9916	0.9877	0.9948	0.9913	0.9992	16.2 s
E-mail	Logistic regression	0.989 ± 0.002	0.9911	0.9870	0.9945	0.9908	0.9992	16.2 s
E-mail	MLP	0.989 ± 0.001	0.9902	0.9884	0.9911	0.9897	0.9992	117.4 s

Source: Own representation of measured experiment outputs (results/e3_models.csv). Training time is for one fit on the full training set.

Cross-validated and test scores agree closely, so the results do not depend on a lucky split (Tab. 5). The linear SVM and logistic regression were the strongest models; across both corpora the SVM had the highest mean cross-validated F1. The neural network did not improve on them: on SMS its F1 was 0.948 against 0.965 for the SVM, with the lowest and most variable cross-validation score, a sign of overfitting on only 4,136 messages. On e-mail it reached 0.990 but needed 7 times as long to train. With thousands of sparse n-gram features, spam and ham are already almost linearly separable, so a hidden layer adds cost without adding accuracy. All models achieved a ROC-AUC above 0.99, meaning their ranking of messages is very good and differences lie mainly in where the decision threshold falls (AUC = area under the ROC curve).

3.4 Generalisation across message types (E4)

Tab. 6 Cross-corpus evaluation (Linear SVM, lemmatised text, TF-IDF)

Trained on	Tested on	Accuracy	Precision	Recall	F1	Ham flagged
SMS	SMS	0.9913	0.9766	0.9542	0.9653	0.33%
SMS	E-mail	0.5444	0.5277	0.4414	0.4807	36.14%
E-mail	SMS	0.3140	0.1448	0.9008	0.2495	77.10%
E-mail	E-mail	0.9916	0.9877	0.9948	0.9913	1.13%
SMS + e-mail	SMS	0.9585	0.7750	0.9466	0.8522	3.98%
SMS + e-mail	E-mail	0.9839	0.9920	0.9742	0.9830	0.72%

Source: Own representation of measured experiment outputs (results/e4_generalisation.csv).

A model is only reliable on the kind of text it was trained on (Tab. 6). The e-mail model flagged 77% of legitimate SMS as spam, because casual text messages look nothing like Enron business mail, and the SMS model detected fewer than half of the spam e-mails. Training on both corpora together largely solved this, but SMS performance still dropped to an F1 of 0.852, and 4.0% of legitimate SMS were flagged, compared with 0.3% for the SMS-only model. The six times larger and almost balanced e-mail corpus dominates the joint model. The final system therefore uses one model per channel.

4. Final system

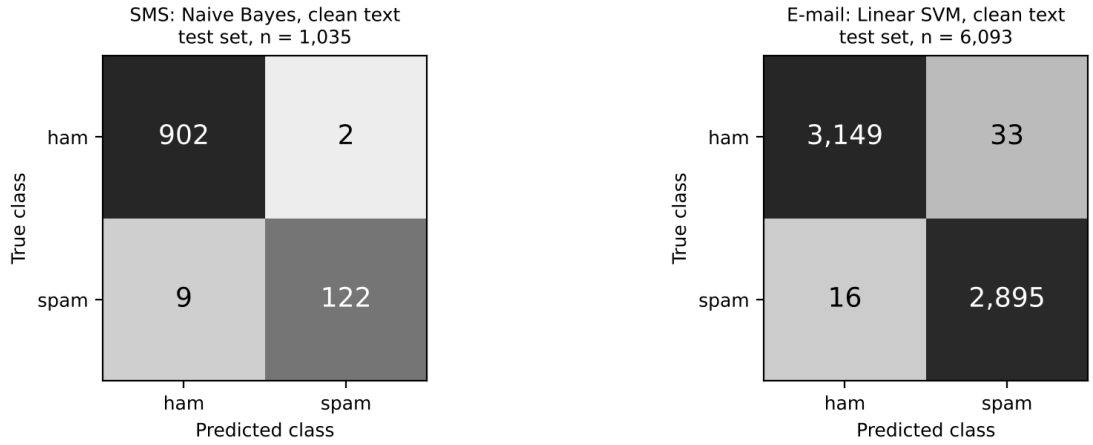
For each channel, all combinations of four preprocessing levels and three classical classifiers were scored by cross-validated F1 on the training data only (5 folds for SMS, 3 for e-mail). The best combination was refitted on the full training set and saved as a pipeline that accepts raw text. A simple router sends multi-line texts or texts longer than 60 words to the e-mail model and all others to the SMS model. The command-line tool *predict.py* classifies new messages with these pipelines. Tab. 7 reports their single evaluation on the untouched test sets.

Tab. 7 Final models on the held-out test sets

	SMS model	E-mail model
Selected by cross-validation	Naive Bayes, clean text	Linear SVM, clean text
Cross-validated F1 (training data)	0.960	0.991
Test messages	1,035	6,093
Accuracy	98.94%	99.20%
Precision / recall	98.39% / 93.13%	98.87% / 99.45%
F1 / ROC-AUC	0.957 / 0.996	0.992 / 0.999
Legitimate messages flagged	2 of 904 (0.22%)	33 of 3,182 (1.04%)
Spam missed	9 of 131	16 of 2,911
Training / prediction time	0.4 s / 0.05 ms per message	30.4 s / 0.86 ms per message

Source: Own representation of measured experiment outputs (results/final_model.json).

Fig. 4 Confusion matrices of the final models



Source: Own representation of measured experiment outputs; rows are true classes, columns predicted classes.

For SMS, cross-validation selected Naive Bayes on lightly cleaned text, without stop-word removal or lemmatising, in line with E1. It flagged only 2 legitimate messages while missing 9 spam messages (Fig. 4). This is the preferable trade-off for a spam filter: a missed spam message is a nuisance, a hidden legitimate one can be costly. Several missed spam messages imitate harmless content, such as a joke ("Latest News! Police station toilet stolen, cops have nothing to go on!") or a notification ("Missed call alert. These numbers called but left no message. 07008009200"). The flagged legitimate messages use product-like wording: "Nokia phone is lovly.." and "Unlimited texts. Limited minutes.". For e-mail, the linear SVM reached an F1 of 0.992. Its strongest spam indicators include your, http, remove, no, software, here, while ham indicators include enron, vince, louise, thanks, attached, energy; part of the signal is thus specific to Enron (Section 5).

5. Discussion and limitations

The experiments show that the value of each pipeline step depends on the kind of text. For short, noisy SMS, normalisation of numbers, currencies and links mattered more than any linguistic processing, and stemming or lemmatising brought no consistent gain. Their main advantage, merging word forms, is small when bigrams and thousands of training examples are available, and they remove stylistic cues such as abbreviations. For long e-mails the choice of preprocessing hardly mattered. Data size mattered more than model choice: going from 100 to 1,000 training messages gained more F1 than any change of classifier at full size. Among the classifiers, discriminative linear models were the best compromise of accuracy, speed and transparency, because their weights directly show which n-grams drive a decision.

The most important finding for deployment is the lack of transfer between message types (E4). A filter is only as good as the match between its training data and the messages it will see, so the system keeps one model per channel and should be retrained on data from its actual environment.

The study has limitations. Enron-Spam consists of e-mails from the early 2000s, and its ham comes from one company; several strong ham indicators are Enron-specific names and terms, so the e-mail scores are likely optimistic for a general mailbox. Modern spam relies more on phishing links, images and several languages. Each corpus was split only once, although cross-validation and repeated subsamples in E2 reduce this risk. The decision threshold was left at the default; in practice it could be tuned to a target false-positive rate. Finally, transformer-based language models were not tested; given the near-perfect ROC-AUC of the linear models, their expected benefit here is small compared with their computational cost, but they could help with more varied data.

6. Conclusion

This project developed and evaluated an NLP system for spam detection. It collects two public labelled corpora, preprocesses the text with normalisation, stop-word removal, stemming or lemmatising, encodes it as Bag-of-Words or TF-IDF n-grams, trains Naive Bayes, a linear SVM, logistic regression and a neural network, and evaluates them from small to large data sets and across message types. Normalising entities was the most useful preprocessing step for SMS (Q1), performance improved quickly up to about 1,000 training messages and then levelled off (Q2), linear models were as accurate as the neural network at much lower cost (Q3), and models did not transfer between SMS and e-mail (Q4). The final channel-specific system classifies new messages with 98.94% (SMS) and 99.20% (e-mail) accuracy on held-out test data while flagging very few legitimate messages. Future work should retrain and test the system on recent, multilingual messages, tune the decision threshold, and compare it with transformer-based models.

References

Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). Association for Computing Machinery. https://doi.org/10.1145/2034691.2034742

Almeida, T. A., & Hidalgo, J. M. G. (2011). *SMS spam collection* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: Textual content based deep learning framework for social spam detection using conjoint attention mechanism. *Journal of Ambient Intelligence and Humanized Computing, 12*(10), 9287–9302. https://doi.org/10.1007/s12652-020-02640-5

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). *Spambase* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C53G6X

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. *Connection Science, 34*(1), 2108–2124. https://doi.org/10.1080/09540091.2022.2098926

Jurafsky, D., & Martin, J. H. (2013). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (2nd ed.). Pearson Prentice Hall.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes – Which naive Bayes? In *Proceedings of the Third Conference on Email and Anti-Spam (CEAS 2006)*. Mountain View, CA, United States.

Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM, 38*(11), 39–41. https://doi.org/10.1145/219717.219748

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Porter, M. F. (1980). An algorithm for suffix stripping. *Program, 14*(3), 130–137. https://doi.org/10.1108/eb046814

Wiechmann, M. (n.d.). *enron_spam_data* [Data set and source code]. GitHub. https://github.com/MWiechmann/enron_spam_data

Code and appendix

The complete source code, tests, results and trained models are available at https://github.com/yousefalashmawi/spam-detection-nlp. The appendix that follows is a PDF rendering of the executed notebook *Spam_Detection_Experiments.ipynb*, which contains every code cell and its observed output; the same notebook is part of the repository.